SpiralTrain
developer courses

Python Libraries

- **Regular Expressions**

- Regular Expressions
- match Function
- Meta Characters
- Quantifiers
- Character Classes
- Rexp Examples
- Matching versus Searching
- Search and Replace
- Regular Expression Modifiers

© copyright : spiraltrain@gmail.com

Regular Expressions

- **Regular expressions are a syntax to match text :**
  - Helps matching strings using a specialized pattern syntax
  - Originated in Perl and available in programming languages like PHP, Java, .NET
- **Usages of Regular Expressions :**
  - Enable to enforce rules on text formats like phone numbers or email addresses
  - Use them to find key data within logs, configuration files or webpages
- **Use them to make complex replacements like :**
  - Find all email addresses in a page and make them address [AT] site [dot] com
- **Module re :**
  - Provides full support for Perl-like regular expressions in Python
  - Exception re.error if an error occurs while compiling or using a regular expression
- **Various characters have special meaning in regular expression :**
  - Avoid confusion while dealing with regular expressions
  - Use Raw Strings as r'expression'

match Function

- **Attempts to match RE pattern to string with optional flags :**

```
re.match(pattern, string, flags=0)
```

- **Parameters :**
  - pattern : regular expression to be matched.
  - String : String which would be searched to match the pattern
  - flags : specify different flags using exclusive OR (|)
- **Returns :**
  - Match object on success
  - None on failure
  - Use group(num) or groups() function of match object to get matched expression
- `group(num=0)` :
  - Returns entire match (or specific subgroup num)
- `groups()` :
  - Return all matching subgroups in a tuple (empty if there weren't any)

Meta Characters

- **Characters with special meaning in regular expressions :**

Metacharacter	Description
.	Matches any character but new line
^	Next character should be at start of string
$	Previous character should be end of string
\	Escapes the next character
\b	Matches on a word boundary
|	Pipe symbol indicates a choice between regular expressions

- **Metacharacters are made non special with a `\` in front of them :**
  - Pattern `/3\.14159/` does not have a wildcard character but a literal dot

Quantifiers

- **Use quantifiers to indicate something must be repeated :**

Quantifier	Description	Example
`?`	zero or one occurrences	`a?`
`*`	zero or more occurrences	`b*`
`+`	one or more occurrences	`c+`
`{min,max}`	at least *min* occurrences, at most *max* occurrences	`{2,5}`
`{equ}`	exactly *equ* occurrences	`{3}`
`{min,}`	at least *min* occurrences	`{2,}`
`{0,max}`	from zero to no more that *max* occurrences	`{0,6}`
`{0,0}`	exactly zero occurrences	`{0,0}`

- **Quantifiers work on the previous item :**
  - Item after which the quantifier occurs
- **Pattern `andromeda\t*nebula` would match :**
  - Word `andromeda` followed by zero or more tabs followed by word `nebula`

Character Classes

- **Indicate list of characters that one element in a string will match :**
  - Inside square brackets "[]" a list of characters can be provided
  - The expression matches if any of these characters is found
  - The order of characters is insignificant
- **Distinguish three kinds of Character Groups :**
  - Indicate which characters may occur
  - Indicate which characters may not occur
  - Indicate which characters are excluded from a group to occur
- **Positive character group :**

```
[ace]  , [A-Za-z]
```

- **Negative character group indicated with ^ inside [and]**

```
[^a-f] , [^KPV]
```

- **Character Class Subtraction :**

```
[A-Z-[QZ]]
```

Regular Expression Examples

Regular Expression	Match
`abc`	This is looking for exact character sequence a, b and then c
`[abc]`	Square brackets match any of characters inside : a, b or c
`(abc)*`	Parenthesis group patterns. Asterix marks zero or more of previous character. Would match empty string or abcabcabc
`\.+`	Backslash is an all purpose escape character. + marks one or more of the previous character. This would match .....
`[0-4]`	Match any number from 0 to 4
`[^0-4]`	Match anything not the number 0-4
`\sword\s`	Match word where there is white space before and after
`\bword\b`	\b marks word boundary. Could be white space, new line or end of the string
`[a-z]{8,}`	Must be at least 8 letters
`\d{3,12}`	\d matches any digit ([0-9]) while the braces mark the min and max count of the previous character. In this case 3 to 12 digits.

Regular Expression Example

```
import re
line = "Cats are smarter than dogs";
matchObj = re.match( r'(.*) are(\.*)', line, re.M|re.I)
if matchObj:
   print ("matchObj.group() : ", matchObj.group())
   print ("matchObj.group(1) : ", matchObj.group(1))
   print ("matchObj.group(2) : ", matchObj.group(2))
else:
   print ("No match!!")
```

- **Result :**

```
matchObj.group(): Cats are
matchObj.group(1) : Cats
matchObj.group(2) :
```

search Function

- **Search for first occurrence of RE pattern within string :**

```
re.string(pattern, string, flags=0)
```

- **Parameters :**
  - pattern : Regular expression to be matched.
  - string : String which would be searched to match the pattern
  - flags : Specify different flags using exclusive OR (|)
- **Returns:**
  - Match object on success
  - None on failure
  - Use group(num) or groups() function of match object to get matched expression

Search Example

```
import re
line = "Cats are smarter than dogs";
matchObj = re.search( r'(.*) are(\.*)', line, re.M|re.I)
if matchObj:
   print ("matchObj.group() : ", matchObj.group())
   print ("matchObj.group(1) : ", matchObj.group(1))
   print ("matchObj.group(2) : ", matchObj.group(2))
else:
   print ("No match!!")
```

- **Result :**

```
matchObj.group(): Cats are
matchObj.group(1) : Cats
matchObj.group(2) :
```

Matching versus Searching

- **Two different operations based on regular expressions :**
  - `match` checks for a match only at the beginning of the string
  - `search` checks for a match anywhere in the string (Perl does as default)

```
import re
line = "Cats are smarter than dogs";
matchObj = re.match( r'dogs', line, re.M|re.I)
if matchObj:
   print ("match --> matchObj.group() : ", matchObj.group())
else: print ("No match!!")
matchObj = re.search( r'dogs', line, re.M|re.I)
if matchObj:
   print ("search --> matchObj.group() : ", matchObj.group())
else: print ("No match!!")
```

- **Result :**

```
No match!!
search --> matchObj.group() :  dogs
```

Search and Replace

- **Important re method that use regular expressions is `sub` :**

```
sub(pattern, repl, string, max=0)
```

- **Replaces all occurrences of RE pattern in string with `repl` :**
  - Substituting all occurrences unless max provided
  - Method would return modified string

```
phone = "2004-959-559 #This is Phone Number"
# Delete Python-style comments
num = re.sub(r'#.*$', "", phone)
print ("Phone Num : ", num)
# Remove anything other than digits
num = re.sub(r'\D', "", phone)
print ("Phone Num : ", num)
```

- **Result :**

```
Phone Num :  2004-959-559
Phone Num :  2004959559
```

Regular Expression Modifiers

- **Optional modifiers control various aspects of matching**
  - Provide multiple modified using exclusive OR (|)

Modifier	Description
re.I	Performs case-insensitive matching.
re.L	Interprets words according to the current locale.This interpretation affects the alphabetic group (\w and \W), as well as word boundary behavior (\b and \B).
re.M	Makes $ match the end of a line (not just the end of the string) and makes ^ match the start of any line (not just the start of the string).
re.S	Makes a period (dot) match any character, including a newline.
re.U	Interprets letters according to the Unicode character set. This flag affects the behavior of \w, \W, \b, \B.
re.X	Permits "cuter" regular expression syntax. It ignores whitespace (except inside a set [] or when escaped by a backslash), and treats unescaped # as a comment marker.

SpiralTrain
developer courses

Python Libraries : Networking

- **Networking**

- Network Layering
- TCP/IP Layering
- TCP/IP Components
- UDP Characteristics
- TCP Characteristics
- Connectionless Services
- Connection Oriented Services
- Socket utility functions
- Handling names and adresses
- Simple Server Limitations
- Asynchronous Servers
- Popular client libraries
- HTTP servers and clients

© copyright : spiraltrain@gmail.com

UDP Characteristics

- **Also datagram-based**
  - Connectionless, unreliable, can broadcast
- **Applications usually message-based**
  - No transport-layer retries
  - Applications handle (or ignore) errors
- **Processes identified by port number**
- **Services live at specific ports**
  - Usually below 1024, requiring privilege

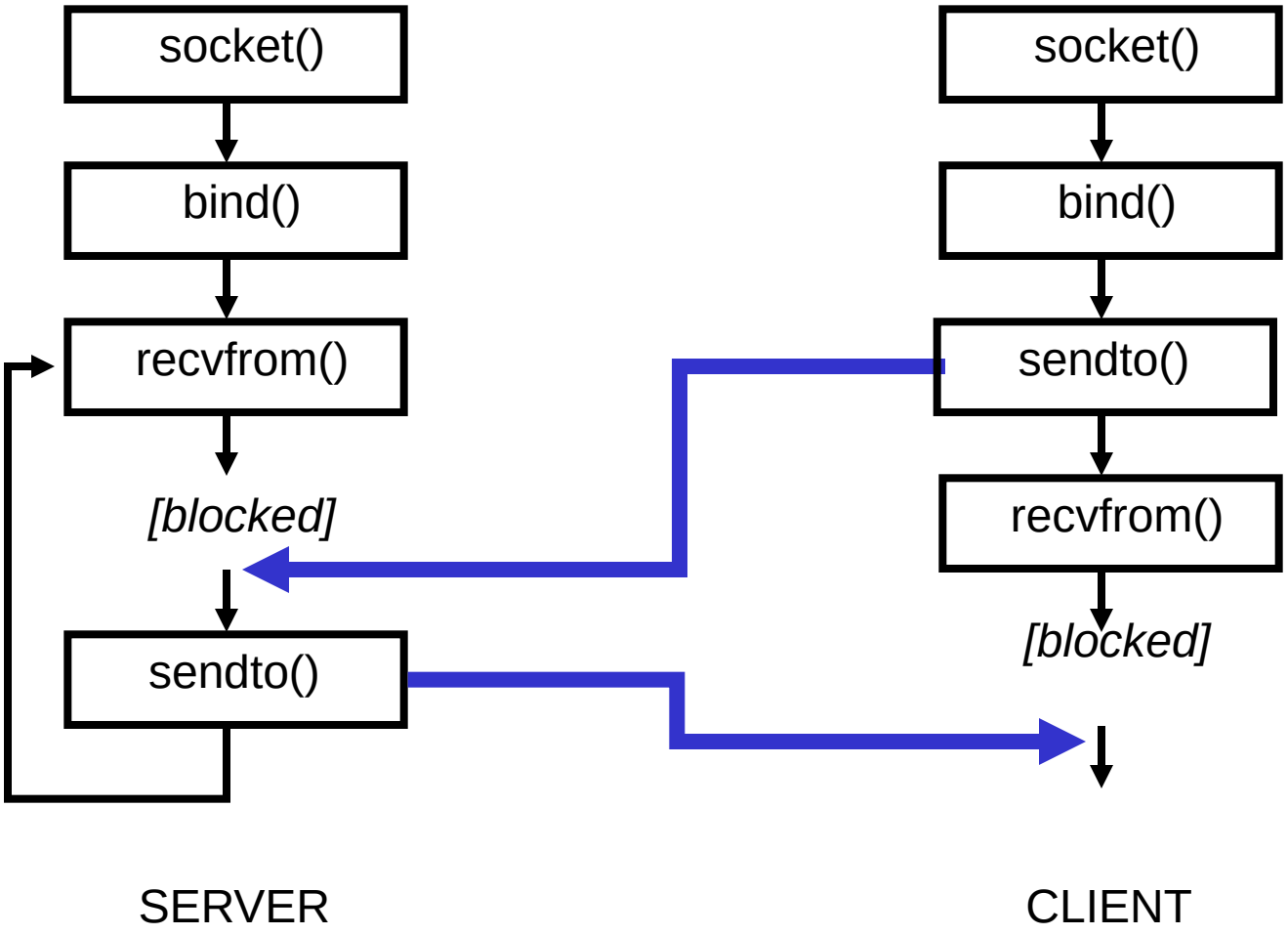
TCP Characteristics

- **Connection-oriented**
  - Two endpoints of a virtual circuit
- **Reliable**
  - Application needs no error checking
- **Stream-based**
  - No predefined blocksize
- **Processes identified by port numbers**
- **Services live at specific ports**

Client/Server Concepts

- **Server opens a specific port**
  - The one associated with its service
  - Then just waits for requests
  - Server is the passive opener
- **Clients get ephemeral ports**
  - Guaranteed unique, 1024 or greater
  - Uses them to communicate with server
  - Client is the active opener

Connectionless Services



Simple Connectionless Server

```
from socket import socket, AF_INET, SOCK_DGRAM
s = socket(AF_INET, SOCK_DGRAM)
s.bind(('127.0.0.1', 11111))
while 1: # nowadays, "while True"
    data, addr = s.recvfrom(1024)
    print "Connection from", addr
    s.sendto(data.upper(), addr)
```

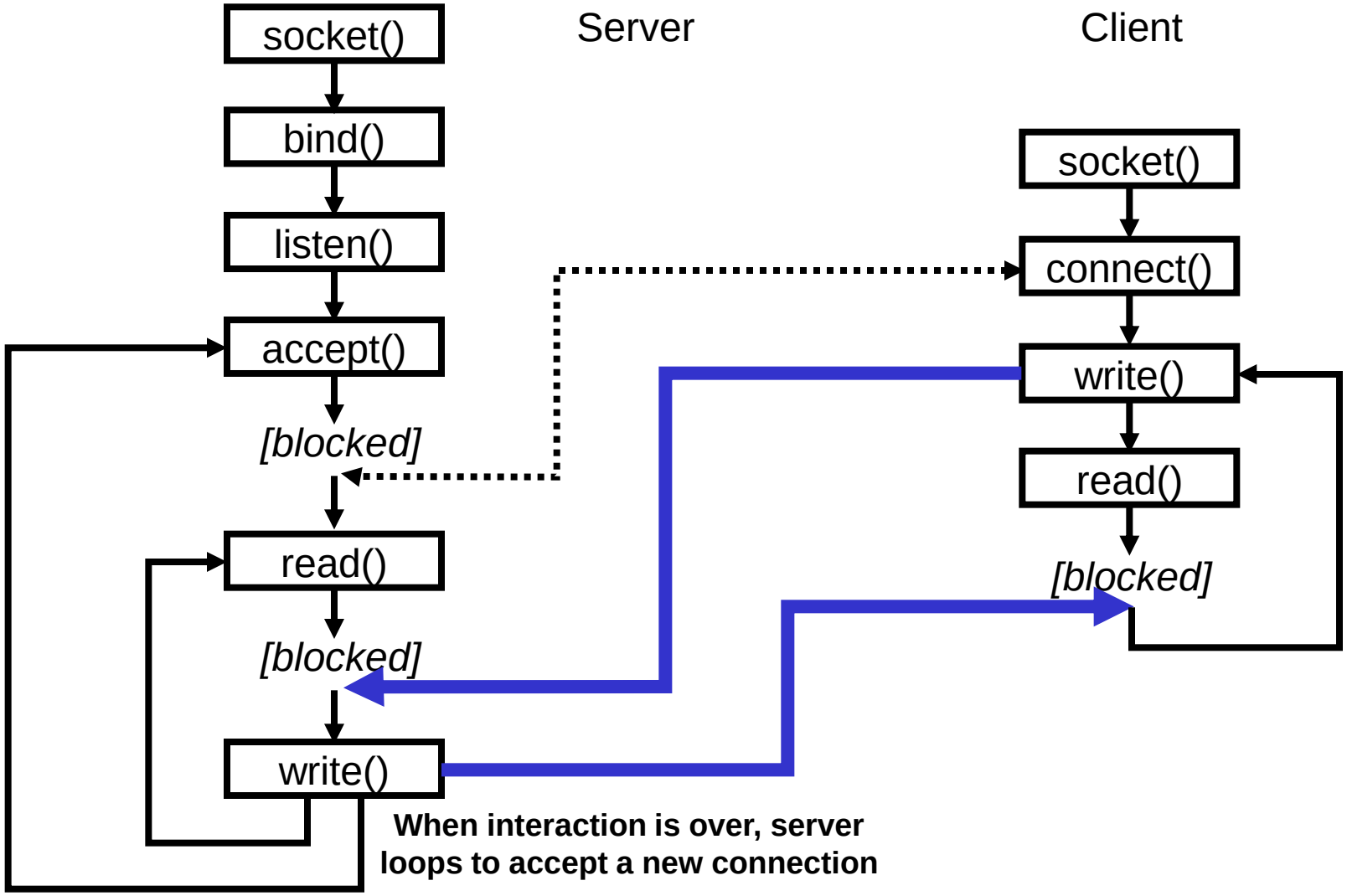
- How much easier does it need to be
- Note that the bind() argument is a two-element tuple of address and port number

Simple Connectionless Client

```
from socket import socket, AF_INET, SOCK_DGRAM
s = socket(AF_INET, SOCK_DGRAM)
s.bind(('127.0.0.1', 0)) # OS chooses port
print "using", s.getsocketname()
server = ('127.0.0.1', 11111)
s.sendto("MixedCaseString", server)
data, addr = s.recvfrom(1024)
print "received", data, "from", addr
s.close()
```

- **Relatively easy to understand?**

Connection-Oriented Services



Connection-Oriented Server

```
from socket import \
      socket, AF_INET, SOCK_STREAM
s = socket(AF_INET, SOCK_STREAM)
s.bind(('127.0.0.1', 9999))
s.listen(5) # max queued connections
while 1:
   sock, addr = s.accept()
    # use socket sock to communicate
    # with client process
```

- **Client connection creates new socket**
  - Returned with address by accept()
- **Server handles one client at a time**

Connection-Oriented Client

```
s = socket(AF_INET, SOCK_STREAM)
s.connect((HOST, PORT))
s.send('Hello, world')
data = s.recv(1024)
s.close()
print 'Received', `data`
```

- **This is a simple example**
  - Sends message, receives response
  - Server receives 0 bytes after close()

Some socket Utility Functions

- `htonl(i), htons(i)` :
  - 32-bit or 16-bit integer to network format
- `ntohl(i), ntohs(i)` :
  - 32-bit or 16-bit integer to host format
- `inet_aton(ipstr), inet_ntoa(packed)` :
  - Convert addresses between regular strings and 4-byte packed strings

Handling Names and Addresses

- `getfqdn(host='')` :
  - Get canonical host name for host
- `gethostbyaddr(ipaddr)` :
  - Returns (hostname, aliases, addresses)
    - Hostname is canonical name
    - Aliases is a list of other names
    - Addresses is a list of IP address strings
- `gethostbyname_ex(hostname)` :
  - Returns same values as `gethostbyaddr()`

Treating Sockets as Files

- `makefile([mode[, bufsize]])` :
  - Creates a file object that references the socket
  - Makes it easier to program to handle data streams
    - No need to assemble stream from buffers

Server Libraries

- **SocketServer module provides basic server features**
- **Subclass `TCPServer` and `UDPServer` classes for specific protocols**
- **Subclass BaseRequestHandler :**
  - Overriding its `handle()` method, to handle requests
- **Mix-in classes allow asynchronous handling**

Using SocketServer Module

- **Server instance created with address and handler-class as arguments:**
- `SocketServer.UDPServer(myaddr,MyHandler)`
- **Each connection/transmission creates a request handler instance by calling the handler-class***
- **Created handler instance handles a message (UDP) or a complete client session (TCP)**
- ***** In Python you instantiate a class by calling it like a function**

Writing a handle() Method

- **self.request gives client access**
  - (string, socket) for UDP servers
  - Connected socket for TCP servers
- **self.client_address is remote address**
- **self.server is server instance**
- **TCP servers should handle a complete client session**

TCP Upper-Case SocketServer

```
# tcps1.py
import SocketServer
class UCHandler(SocketServer.BaseRequestHandler):
   def handle(self):
      print "Connected:", self.client_address
      while 1:
         data = self.request.recv(1024)
         if data == "\r\n":
            break
            print data[:-2]
         self.request.send(data.upper())
myaddr = ('127.0.0.1', 2345)
myserver = SocketServer.TCPServer(myaddr, UCHandler)
myserver.serve_forever()
```

Change this function to alter server's functionality

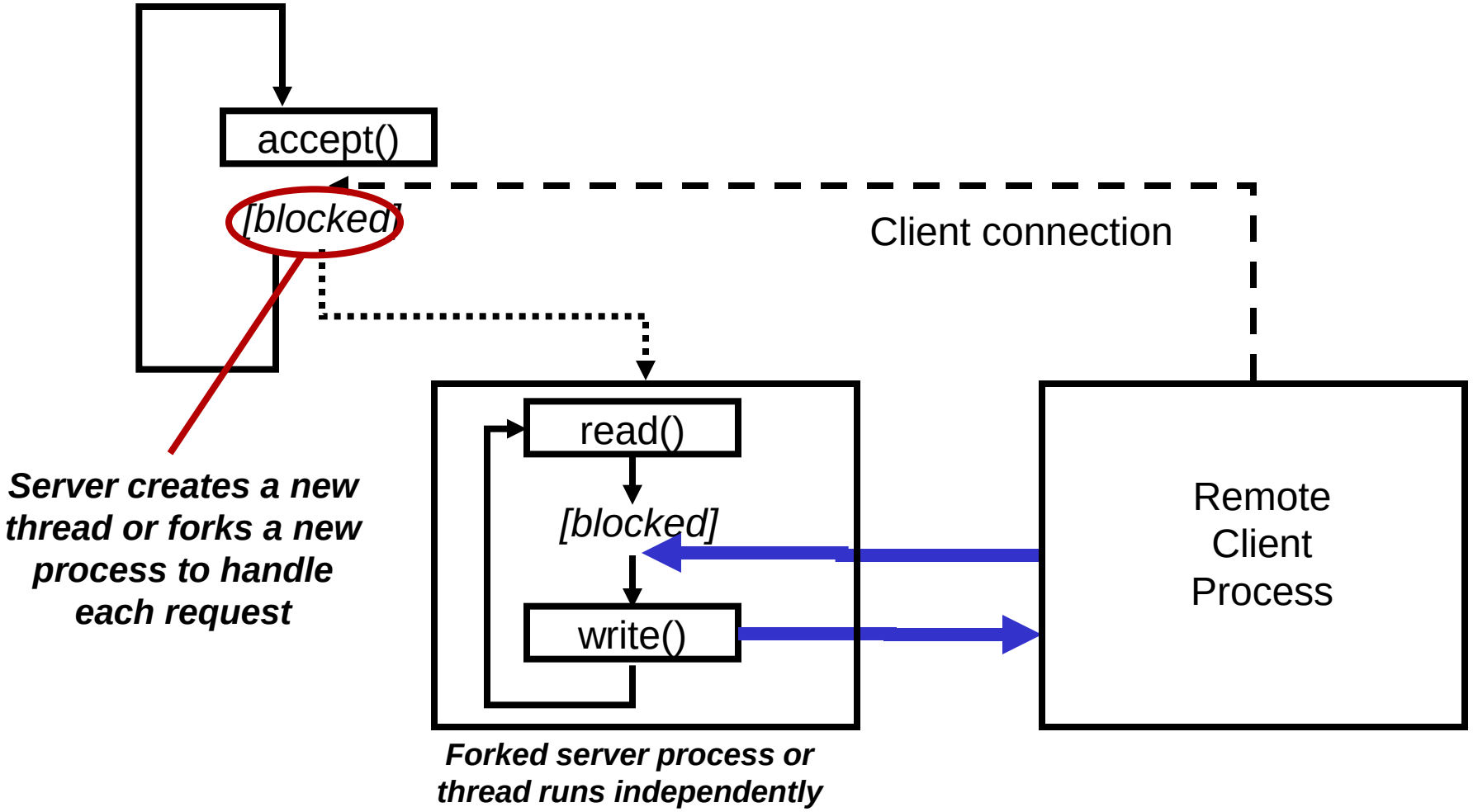
TCP Upper-Case Client

```
# tcpc1.py
from socket import socket, AF_INET, SOCK_STREAM
srvaddr = ('127.0.0.1', 2345)
s = socket(AF_INET, SOCK_STREAM)
s.connect(srvaddr)
while 1:
    data = raw_input("Send: ")
    s.send(data + "\r\n")
    if data == "":
        break
    data = s.recv(1024)
    print data[:-2] # Avoids doubling-up the newline
s.close()
```

Skeleton Server Limitations (1)

- **UDP server adequate for short requests**
  - If service is extended, other clients must wait
- **TCP server cannot handle concurrent sessions**
  - Transport layer queues max 5 connections
    - After that requests are refused
- **Solutions?**
  - Fork a process to handle requests, or
  - Start a thread to handle requests

Simple Server Limitations (2)



Asynchronous Server Classes

- **Use provided asynchronous classes :**

```
myserver = SocketServer.TCPServer(myaddr, UCHandler)
```

- **becomes :**

```
myserver = SocketServer.ThreadingTCPServer(myaddr, UCHandler)
```

- **or**

```
myserver = SocketServer.ForkingTCPServer(myaddr, UCHandler)
```

SpiralTrain
developer courses

Python Libraries : Unit Testing

- **Unit Testing**

- What is Unit Testing?
- Benefits of Unit Testing
- Manual Testing
- Automated Testing
- Time to Test
- Unit Test Example
- Unit Testing Best Practices
- Testing Frameworks

© copyright : spiraltrain@gmail.com

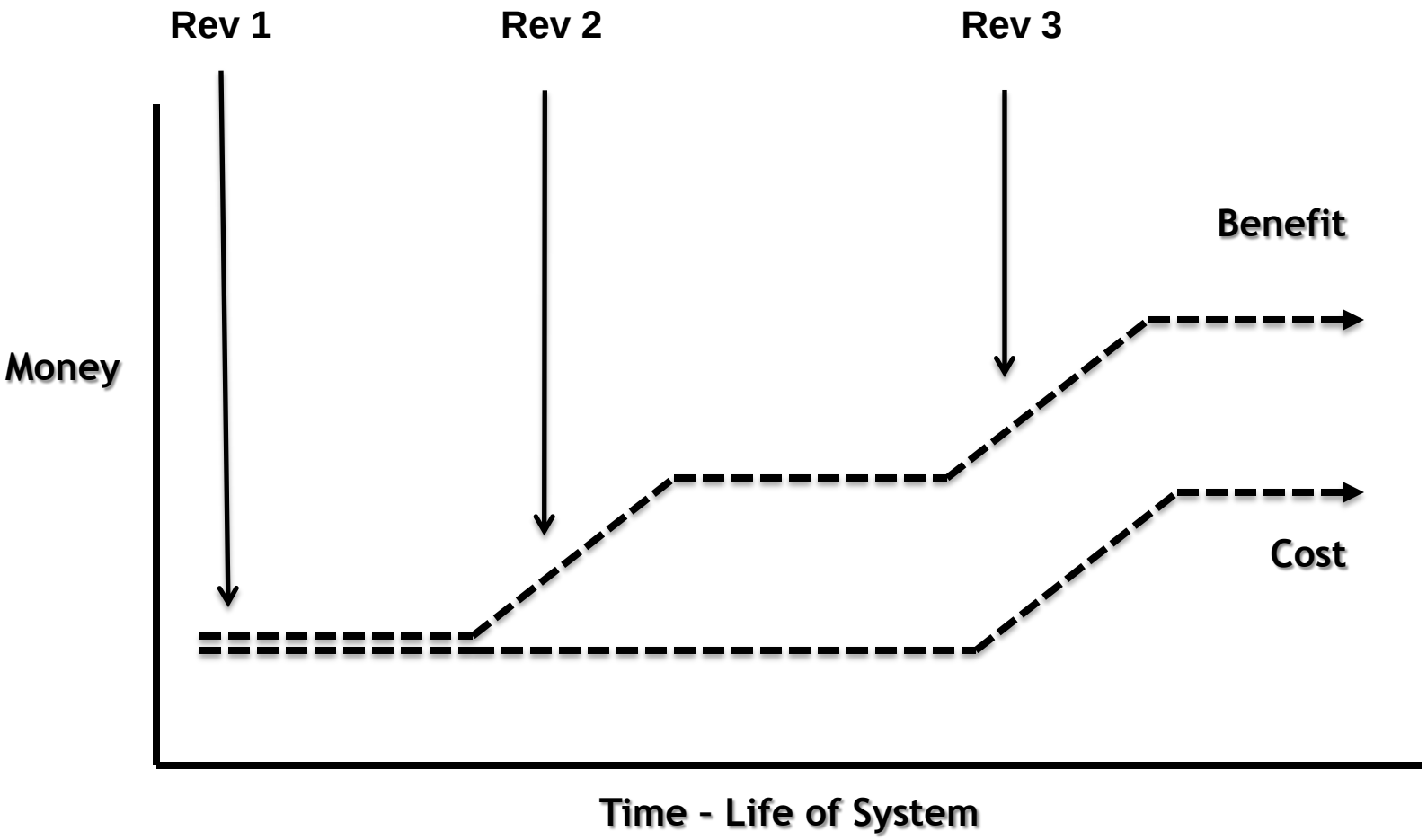
What is Unit Testing?

- **Testing in general checks application functionality :**
  - Whether application fulfills the requirements
- **Definition of Unit Testing :**
  - Method of testing that verifies that individual units of code are working properly
- **Unit is smallest testable part of an application :**
  - In procedural programming unit may be function, procedure, web page, menu etc.
  - In object-oriented programming smallest unit is always a class
- **Unit testing is a developer's job :**
  - Involves writing and refactoring code
- **Unit Testing does not touch anything external :**
  - Database, file, network, Web Server
- **Unit Testing can be done manually or automated :**
  - It is essential to ensure quality to customers
- **Unit Testing is cornerstone in Agile Development**

Benefits of Unit Testing

- **Unit Tests increase their value over time :**
  - Allows others to prove the software still works as tested
- **Unit Tests provide a documentation of requirements :**
  - Executable, evolving documentation of how the code works
- **Gives confidence and sleep to developers :**
  - Allows code to be less brittle allowing for refactoring without fear
- **Improves the quality of the code with fewer bugs :**
  - Automated regression tests, ensure code continues to work
- **Unit Tests lead to smaller, tighter and decoupled code :**
  - Code is tested in isolation of the rest of the system which may not be written yet
- **Integration testing becomes much easier :**
  - By testing parts of a program first and then testing the sum of its parts

Benefits of Unit Testing



Manual Testing

- **Unit Tests can be created and run manually without tool support :**
  - Since test cases are executed by humans it is time consuming and tedious
- **Huge investment in human resources :**
  - More testers are required
- **Less reliable :**
  - Tests may not be performed with precision each time because of human errors
- **Non-programmable :**
  - No programming can be done to write sophisticated tests which fetch hidden data
- **Typically manual testing should be avoided :**
  - Is only useful in those cases when one can't make automated tests
  - Typically such test are at a higher level : e.g. integration, GUI, stress etc. tests

Automated Testing

- **Executing test cases by using automation tool :**
  - Automated tests using a test framework prove features
- **Fast :**
  - Automation runs test cases significantly faster than human resources
- **Less investment in human resources :**
  - Less testers are required in automation testing
- **More reliable :**
  - Automation tests perform precisely same operation each time they are run
- **Programmable :**
  - Testers can program sophisticated tests to bring out hidden information
- **Most teams automatically run tests :**
  - As part of the continuous integration process

Time to Test

Before coding
Test Driven Development

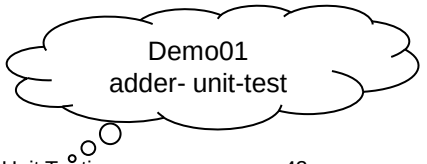
- Focus on requirements
- Thinking about how code will be consumed
- Stop coding when reqs met
- Harder initially

After/During coding

- Focus on code
- Thinking about algorithm
- More refactoring
- Easier initially

Unit Test Example

```
interface Adder {
     int add(int a, int b);
}
class AdderImpl implements Adder {
    public int add(int a, int b) {
        return a + b;
    }
}
public class TestAdder {
@Test
    public void testSum() {
        Adder adder = new AdderImpl();
        assertTrue(adder.add(1, 1) == 2);     // can it add positive numbers?
        assertTrue(adder.add(0, 0) == 0);     // is zero neutral?
        assertTrue(adder.add(-1, -2) == -3); // can it add negative numbers?
        assertTrue(adder.add(-1, 1) == 0);    // positive and  negative?
        assertTrue(adder.add(1234, 988) == 222); // larger numbers?
    }
}
```



Unit Testing Best Practices

- **Unit tests should run fast and automatically on check in :**
  - Unit tests should be developed in parallel to the code
- **Tests should have one logical assertion per test :**
  - Counterproductive to `assert()` anything that's also asserted by another test
- **Unit tests should mock out all external services :**
  - Code to abstractions, wrap difficult code, use mocking tool for dependencies
- **Mock out all state :**
  - Keeping state means unit tests can influence each other's outcome
- **Test only one code unit at a time :**
  - Classes or very small groups of classes
- **Using continuous integration software :**
  - Provides quick feedback for any integration problems
- **Unit Testing Don'ts :**
  - Ignore failing tests, test code you didn't write, overuse Setup or Teardown

Testing Frameworks

- **Unit Test typically are executed using a Unit Test Framework :**
  - JUnit (Java) : `http://www.junit.org/`
  - TestNG (Java) : `http://testng.org/`
  - JDave (Java) : `http://www.jdave.org/`
  - JBehave (Java) : `http://jbehave.org/`
- **Considerations when choosing a Unit Test Framework :**
  - Does it integrate well in my development environment?
  - Does it integrate with my preferred mocking frameworks?
  - Is it possible to run tests in parallel, on multiple CPU cores?
- **Comparing JUnit and TestNG :**
  - Facility to rerun failed tests is very helpful is available only in TestNG
  - JUnit has most tool support with JUnit test runners appearing in many places
  - HTML reports generated by JUnit are really good
  - TestNG is also capable of handling more complex tests
  - JUnit 4.x was later released with features similar to those found on TestNG

Python Libraries : Date and Time

- **Date and Time**

- Date and Time Handling
- Time Tuple
- struct-time
- Getting Current Time
- Getting Formatted Time
- Getting Calendar for month
- time Module
- Calendar Functions

Date and Time Handling

- **Python's time and calendar modules help track dates and times :**
  - Converting between date formats is a common chore
- **Tick :**
  - Time intervals are floating-point numbers in units of seconds
  - Instants in time expressed in seconds since 12:00am, January 1, 1970(epoch)
- **Time module :**
  - Has functions for working with times, and for converting between representations
  - `time.time()` returns system time in ticks since 12:00am, Jan 1, 1970(epoch)

```
import time;  # This is required to include time module.
ticks = time.time()
print ("Ticks since 12:00am, January 1, 1970:", ticks)
```

- **Result :** `Ticks since 12:00am, January 1, 1970: 7186862.73399`
- **Date arithmetic is easy to do with ticks**
  - Dates before the epoch cannot be represented in this form
  - Dates in the far future also cannot be represented this way
  - Cutoff point is sometime in 2038 for UNIX and Windows

TimeTuple

- **Many time functions handle time as tuple of 9 numbers**

Index	Field	Values
0	4-digit year	2008
1	Month	1 to 12
2	Day	1 to 31
3	Hour	0 to 23
4	Minute	0 to 59
5	Second	0 to 61 (60 or 61 are leap-seconds)
6	Day of Week	0 to 6 (0 is Monday)
7	Day of year	1 to 366 (Julian day)
8	Daylight savings	-1, 0, 1, -1 means library determines DST

struct_time structure

- **TimeTuple is equivalent to struct_time structure**

Index	Attributes	Values
0	tm_year	2008
1	tm_mon	1 to 12
2	tm_mday	1 to 31
3	tm_hour	0 to 23
4	tm_min	0 to 59
5	tm_sec	0 to 61 (60 or 61 are leap-seconds)
6	tm_wday	0 to 6 (0 is Monday)
7	tm_yday	1 to 366 (Julian day)
8	tm_isdst	-1, 0, 1, -1 means library determines DST

Getting Current Time

- **Translate a time instant :**
  - From a seconds since the epoch floating-point value into a time-tuple
  - Pass floating-point value to function that returns time-tuple with all nine items valid

```
localtime = time.localtime(time.time())
print ("Local current time :", localtime)
```

- **Result :**

```
Local current time : (2008, 5, 15, 12, 55, 32, 0, 136, 1)
```

- **Result could be formatted in any other presentable form**

Getting Formatted Time

- **Simple method to get time in readable format is `asctime()` :**

```
import time;
localtime = time.asctime( time.localtime(time.time()) )
print ("Local current time :", localtime)
```

- **Result :**

```
Local current time : Tue Jan 13 10:17:09 2009
```

Getting calendar for a month

- **Calendar module :**
  - Gives a wide range of methods to play with yearly and monthly calendars
- **Print a calendar for a given month (Jan 2008) :**

```
import calendar
cal = calendar.month(2008, 1)
print ("Here is the calendar:")
print (cal)
```

- **Result :**

```
    January 2008
Mo Tu We Th Fr Sa Su
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30 31
```

time Module

- **Has functions for working with times for converting**
- `time.altzone` :
  - Offset of the local DST timezone, in seconds west of UTC, if one is defined
  - Negative if local DST timezone is east of UTC (as in Western Europe, including UK)
  - Only use this if daylight is nonzero.
- `time.asctime([tupletime])` :
  - Accepts a time-tuple
  - Returns a readable 24-character string such as 'Tue Dec 11 18:07:14 2008'
- `time.clock()` :
  - Returns current CPU time as a floating-point number of seconds
  - To measure computational costs of different approaches :
    - value of time.clock is more useful than that of time.time().
- `time.ctime([secs])` :
  - Like asctime(localtime(secs)) and without arguments is like asctime( )

Attribute with Description

- `time.localtime([secs])` :
  - Accepts an instant expressed in seconds since the epoch and returns a time-tuple t with the local time (t.tm_isdst is 0 or 1, depending on whether DST applies to instant secs by local rules).
- `time.mktime(tupletime)` :
  - Accepts an instant expressed as a time-tuple in local time and returns a floating-point value with the instant expressed in seconds since the epoch.
- `time.sleep(secs)` :
  - Suspends the calling thread for secs seconds.
- `time.strftime(fmt[,tupletime])` :
  - Accepts an instant expressed as a time-tuple in local time and returns a string representing the instant as specified by string fmt.
- `time.strptime(str,fmt='%a %b %d %H:%M:%S %Y')`
  - Parses str according to format string fmt and returns the instant in time-tuple format.

calendar Module

- **Supplies calendar-related functions :**
  - Including functions to print a text calendar for a given month or year
- **By default :**
  - calendar takes Monday as the first day of the week
  - Sunday as the last one
  - Change this, call calendar.setfirstweekday() function

Calendar Functions

- `calendar.calendar(year,w=2,l=1,c=6)` :
  - Returns a multiline string with a calendar for year year formatted into three columns separated by c spaces. w is the width in characters of each date; each line has length 21*w+18+2*c. l is the number of lines for each week.
- `calendar.firstweekday( )` :
  - Returns the current setting for the weekday that starts each week. By default, when calendar is first imported, this is 0, meaning Monday.
- `calendar.isleap(year)` :
  - Returns True if year is a leap year; otherwise, False.
  - calendar.leapdays(y1,y2) :
  - Returns the total number of leap days in the years within range(y1,y2).
- `calendar.month(year,month,w=2,l=1)` :
  - Returns a multiline string with a calendar for month month of year year, one line per week plus two header lines. w is the width in characters of each date; each line has length 7*w+6. l is the number of lines for each week.

Calendar Functions

- `calendar.monthrange(year,month)` :
  - Returns two integers. The first one is the code of the weekday for the first day of the month month in year year; the second one is the number of days in the month. Weekday codes are 0 (Monday) to 6 (Sunday); month numbers are 1 to 12.
- `calendar.prcal(year,w=2,l=1,c=6)` :
  - Like print calendar.calendar(year,w,l,c).
- `calendar.prmonth(year,month,w=2,l=1)` :
  - Like print calendar.month(year,month,w,l).
- `calendar.setfirstweekday(weekday)` :
  - Sets the first day of each week to weekday code weekday. Weekday codes are 0 (Monday) to 6 (Sunday).
- `calendar.timegm(tupletime)` :
  - The inverse of time.gmtime: accepts a time instant in time-tuple form and returns the same instant as a floating-point number of seconds since the epoch.

Other Modules Functions:

- **Other important modules and functions with date and time :**

- `datetime Module`
- `pytz Module`
- `dateutil Module`